

This homework will study the optimization of the following two functions $f_1(x)$ and $f_2(x)$:

$$f_1(x) = \frac{1}{2}x^tAx - b^tx + c$$

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad A = \begin{bmatrix} 3 & 2 \\ 2 & 6 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ -8 \end{bmatrix}, \quad c = 0$$

$$f_2(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2 \quad (\text{Rosenbrock's function})$$

- Find the extrema of these functions using calculus. Find the minima

$$f_1(x)' = Ax - b$$

Set $f_1(x)' = 0$ minima is from solving $Ax = b$ and the answer is $x_1 = 2$ and $x_2 = -2$.

$$f_2(x)' = [-400(x_2 - x_1^2) * x_1 - 2(1 - x_1); 200(x_2 - x_1^2)]$$

Set $f_2(x)' = 0$ the answer is $x_1 = 1$ and $x_2 = 1$.

- Compute the Jacobian and Hessian of each function. Write a Matlab function that computes these.

$$J(f_1) = \begin{bmatrix} 3x_1 + 2x_2 - 2 \\ 2x_1 + 6x_2 + 8 \end{bmatrix}$$

$$J(f_2) = \begin{bmatrix} -400(x_2 - x_1^2) * x_1 - 2(1 - x_1) \\ 200(x_2 - x_1^2) \end{bmatrix}$$

$$H(f_1) = A$$

$$H(f_2) = \begin{bmatrix} -400(x_2 - 3x_1^3) + 2 & -400x_1 \\ -400x_1 & 200 \end{bmatrix}$$

```
function [ v,j,h ] = Objective_f1( x )
%OBJECTIVE_F1 Objective function, gradient and hessian of the
quadratic
%function f_1
A = [3 2;2 6];
b = [2;-8];
c = 0;
v = (1 / 2) * (x' * A * x) - b' * x + c;
j = [3*x(1) + 2*x(2) - 2;2*x(1) + 6*x(2) + 8];
h = A;
end
```

```
function [ v,j,h ] = Objective_f2( x )
%OBJECTIVE_F2 Objective function, gradient and hessian of
Rosenbrock's
%function
v = 100*(x(2)-x(1).^2).^2+(1-x(1)).^2;
j = [-400*(x(2)-x(1)^2)*x(1)-2*(1-x(1)) ; 200*(x(2)-x(1)^2)];
h = [-400*(x(2)-3*x(1)^2)+2, -400*x(1); -400*x(1), 200];
end
```

- Write a function for implementing the method of steepest descent with a specified tolerance tol and a maximum number of iterations itmax . You can use the built-in Matlab

function fzero for line minimization.

```
function [Xval, Fval] = steepest_descent(f, x0, tol, itmax)
%STEEPEST_DESCENT
% f is a function handle
% x0 is an initial guess
% tol is the tolerance
% itmax is the maximum number of iterations
xold = x0 + tol + 1;
x = x0;
ite = 0;
while((sum(abs(x - xold)) > tol) && (ite < itmax))
    ite = ite + 1;
    [~,g,~] = f(x);
    xold = x;
    alpha = 1;
    alpha = linsearch(x, alpha, -g, f);
    x = x - alpha * g;
end
Xval = x;
[Fval,~,~] = f(x);

function lambda = linsearch(X, lambda, D, myFx)

    MaxIt = 100;
    Toler = 0.000001;

    iter = 0;
    bGoOn = true;
    while bGoOn
        iter = iter + 1;
        if iter > MaxIt
            lambda = 0;
            break
        end

        h = 0.01 * (1 + abs(lambda));
        f0 = myFxEx(X, D, lambda, myFx);
        fp = myFxEx(X, D, lambda+h, myFx);
        fm = myFxEx(X, D, lambda-h, myFx);
        deriv1 = (fp - fm) / 2 / h;
        deriv2 = (fp - 2 * f0 + fm) / h ^ 2;
        diff = deriv1 / deriv2;
        lambda = lambda - diff;
        if abs(diff) < Toler
            bGoOn = false;
        end
    end

function y = myFxEx(X, DeltaX, lambda, myFx)

    X = X + lambda * DeltaX;
    y = feval(myFx, X);
```

4. Compute using the method of steepest descent, setting tol to 10^{-6} and itmax to 100, the minimum for f1 and f2, and an initial guess of $[1, 0.3863]^T$

```
%% initialization
tol = 10 ^ -6;
itmax = 100;
x0 = [1;0.3863];

%% function 1
f = @Objective_f1;
[Xval, Fval] = steepest_descent(f, x0, tol, itmax)

%% function 2
f = @Objective_f2;
[Xval, Fval] = steepest_descent(f, x0, tol, itmax)
```

For function 1, the output is as follows.

```
%% function 1
Xval =

    2.0000
   -2.0000

Fval =

  -10.0000
```

Steepest descent correctly recovers the true minima of the quadratic function.

And for function 2, the output is as follows.

```
%% function 2
Xval =

    1.0000
    0.3863

Fval =

   37.6628
```

Steepest descent cannot recover the true minima of the Rosenbrock's function because of the small valleys which cause divergence. The result is not diverged but just wandering near the true minima since the gradient is orthogonal to the direction to the true minima.

5. For the Rosenbrock function, implement Newton's method starting from the same guess. Take full steps, without line search. I use learning rate = 1 for the full newton step.

```

function [Xval, Fval] = newton(f, x0, tol, itmax)
%NEWTON
% f is a function handle
% x0 is an initial guess
% tol is the tolerance
% itmax is the maximum number of iterations
xold = x0 + tol + 1;
x = x0;
ite = 0;
while((sum(abs(x - xold)) > tol) && (ite < itmax))
    ite = ite + 1;
    [~,g,h] = f(x);
    xold = x;
    x = x - h \ g;
end
Xval = x;
[Fval,~,~] = f(x);
end

```

The newton method can recover the true minima of the Rosenbrock's function.

```

%% function 2 newton
Xval =
    1
    1

Fval =
    0

```

6. For the Rosenbrock function, implement the limited memory Davidson Fletcher Powell quasi-Newton method starting from the same guess. Use the update formula

$$C^{(k+1)} = C^{(k)} - \frac{C^{(k)} y^{(k)} y^{(k)T} C^{(k)}}{y^{(k)T} y^{(k)}} + \frac{s^{(k)} s^{(k)T}}{y^{(k)T} s^{(k)}}, \quad C^{(0)} = I$$

- Consider $k=2$. What vectors would you store to be able to form $C^{(3)}\mathbf{v}$ for an arbitrary vector $\mathbf{v}$?
- How many operations and how much memory would it take to form $C^{(3)}\mathbf{v}$?
- Do by hand two iterations for the Rosenbrock function starting from a guess $[1,1]^T$

The implementation is as follows.

```

function [Xval, Fval] = DFP(f, x0, tol, itmax)
%DFP
% f is a function handle
% x0 is an initial guess
% tol is the tolerance
% itmax is the maximum number of iterations
xold = x0 + tol + 1;
x = x0;
ite = 0;
C = eye(length(x));
while((sum(abs(x - xold)) > tol) && (ite < itmax))
    ite = ite + 1;
    [~,g,~] = f(x);
    xold = x;
    d = -C * g;
    x = x + d;
    % values to store
    [~,g_1,~] = f(x);
    y = g_1 - g;
    s = d;
    Cy = C * y;
    y_Cy = y' * C * y;
    y_s = y' * s;
    % update C^(k)
    C = C - ((Cy * Cy') / (y_Cy)) + ((s * s') / (y_s));
end
Xval = x;
[Fval,~,~] = f(x);
end

```

- a) $C^{(0)}y^{(0)}, s^{(0)}, C^{(1)}y^{(1)}, s^{(1)}, C^{(2)}y^{(2)}, s^{(2)}$ and 6 scalar values $y^{(0)T}C^{(0)}y^{(0)}, y^{(0)T}s^{(0)}, y^{(0)T}C^{(1)}y^{(1)}, y^{(0)T}s^{(1)}, y^{(0)T}C^{(2)}y^{(2)}, y^{(0)T}s^{(2)}$.
- b) $O(n)$ operations and $6n + 6$ storage where n is the length of an arbitrary vector v .
- c) $x_0 = [1, -1]^T$
 First iteration,
 $x = [1 \ -1]$
 $g = [800 \ -400]$
 $C = [1 \ 0; 0 \ 1]$
 $d = [-800 \ 400]$
 $x = [-799 \ 399]$ (update x)
 $y = e+11 * [-2.0391; -0.0013]$
 $s = [-800 \ 400]$
 $C = [0 \ -0.0006; -0.0006 \ 1]$ (update C)
 Second iteration,
 $x = [-799 \ 399]$
 $g = e+11 * [-2.0391; -0.0013]$
 $C = [0 \ -0.0006; -0.0006 \ 1]$
 $d = e+3[1.0007; 1.4014]$
 $x = e+3[0.2017 \ 1.8004]$ (update x)
 $y = e+11 * [2.0704; 0.0012]$
 $s = e+3[1.0007; 1.4014]$
 $C = [0 \ -0.0004; -0.0004 \ 0.6895]$ (update C)

7. For the Rosenbrock function, using the method of Lagrange multipliers, compute by hand the constrained minimum, subject to the constraint

$$x_1^2 + x_2^2 = 1.$$

$$f_2(x) = 100(x_2 - x_1^2)^2 + (1 - x_1)^2$$

$$f_2(x)' = [-400(x_2 - x_1^2) * x_1 - 2(1 - x_1); 200(x_2 - x_1^2)]$$

$$L(x, \lambda) = f_2(x) - \lambda^T (x_1^2 + x_2^2 - 1)$$

Set the first derivative of $L(x, \lambda)$ to zero,

$$(x_1^2 + x_2^2 - 1) = 0$$

$$[-400(x_2 - x_1^2) * x_1 - 2(1 - x_1) - 2\lambda^T x_1; 200(x_2 - x_1^2) - 2\lambda^T x_2] = 0$$

the answer is $x_1 = 0.7864$ and $x_2 = 0.6177$.